1 Lesson 21-10-26

1.1 Context switching

This is likely better explained in code.

1.2 Real-time scheduling

Let's now focus on how real-time systems define **schedulers**.

We first focus on what *real-time* means. Most programs only have to be correct: given the input, compute the right output, whenever it is ready. A real-time system must be correct and *on time*: it reacts to an event from the outside world, and the result must be ready within a fixed time, the deadline.

We saw there were categories of real-time systems:

- **Hard**: if the task deadline is missed, catastrophic consequences on the system or its environment;
- **Firm**: the result is useless and is thrown away, but no serious damage;
- **Soft**: the result is still useful, only worse; meeting the deadline is desirable.

A fast system minimises the average time a set of tasks takes: (average turnaround, average waiting). A real-time system guarantees the timing of each task: every job must finish by its own deadline, even in the worst case. A low average says nothing about one task: one late job can hide behind many early ones. A faster CPU helps, but it is not a guarantee: what matters is which task runs when, that is the scheduling. So a real-time system values **predictability**: we must be able to compute, in advance, when each task finishes.

1.2.1 Types of tasks

Real-time system discussion benefits from a quick overview on **task types**. There are three:

- **Periodic**: a new job is released at a constant rate, every period T. Also called time-driven. Easy to reason about (and do maths on), also quite similar to how most embedded systems are structured: every T seconds the system should for instance check input, execute an algorithm, update a value, repeat ...
- **Aperiodic**: jobs are released by events, at irregular times, with no known minimum gap: in theory two can arrive at the same instant. This is harder to handle than the one above;
- **Sporadic**: aperiodic, but two consecutive jobs are always at least a minimum inter-arrival time apart. In the worst case, a sporadic task behaves like a periodic task with T. These are easier to reason about if we assume them to not be that frequent and lift the hard real-time assumption slightly.

We will use a simplified model for tasks.

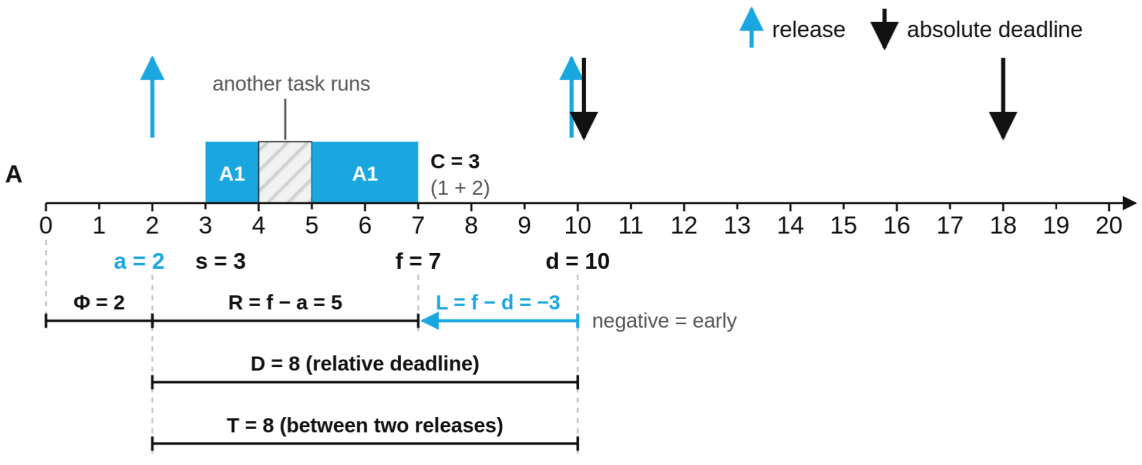
- We have a *fixed number* of tasks, all known in advance;
- Every task is periodic, with a known period T;

- The tasks are independent: they share no resources and do not wait for each other (no precedence);
- Each task has a hard deadline equal to its period: a job must finish before the next job of the same task is released ($D = T$);
- Each task has a known, fixed worst-case execution time C. This can be obtained in practice via either static measuring (before uploading programs to the OS) or dynamic measuring (by the OS itself).
- There is one CPU, and all the kernel's overheads, for example the context switch, are negligible.

1.2.2 Scheduling metrics

Let's introduce some **metrics**.

- **Release** (arrival, activation) time a: the job becomes READY. Job k is released at $a = \phi + (k - 1) \times T$;
- **Phase** ϕ : the release time of the first job. $\phi = 0$ if not given;
- **Computation** time C: the CPU time the job needs; we use its worst case, the **WCET** (*Worst-Case Execution Time*);
- **Period** T: the time between two releases.
- **Relative deadline** D the time the job has after its release; $D = T$ here;
- **Absolute deadline** $d = a + D$: the instant the job must be finished by. With $D = T$, it is the next release;
- **Start time** s, **finish time** f. In general $f \neq s + C$, because the job can be preempted;
- **Response** time $R = f - a$: from release to the end. A job meets its deadline when $R \leq D$;
- **Lateness** $L = f - d$: negative means early, 0 means just in time, positive means deadline missed.



The response time $R = f - a$ is sometimes called *turnaround*, and $s - a$ the response time. In real-time systems, we stick to the above convention.

It is also extremely useful to define the **utilization** of the CPU as:

$$U = \sum_{i}^{N} \frac{C_i}{T_i}$$

That is the sum of all execution time over period ratios for all tasks.

Lastly, we define the hyperperiod H as the least common multiple of all the task periods. With all phases at 0, every task releases a job at 0 and again at H, so the pattern of releases repeats at every H.

1.2.3 Timer scheduling

Some embedded applications are simple: a few periodic tasks that do not interact, for example reading sensors through one shared ADC and updating outputs. They do not need a full operating system. **Timeline** scheduling, also called a cyclic executive: the whole schedule is computed offline, before the system runs, and stored as a fixed table. The time axis is divided into slots of equal length, the *minor cycles* (or frames), equal to the greatest common divisor of the periods. Each frame runs a fixed list of jobs, one after the other, without preemption, and a timer interrupt marks the start of every frame. The sequence of frames that then repeats identically is the *major cycle*, equal to the least common divisor. At run time there is no scheduler deciding anything: the program follows the table.

